



Université Mohammed Premier
École Nationale des Sciences Appliquées d'Oujda
Filière Génie Informatique – GI3

Rapport de Mini-Projet

[GESTION D'UNE CLINIQUE]

Réalisé par :

Souaad Marjij
Rayhane Mokhtari

Encadrante :

Mme Douae EL HILA

Module :

Développement Java IHM

Année académique :

2025/2026

Remerciements

Au terme de ce projet, nous tenons à exprimer notre profonde gratitude et nos sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

Nous adressons tout particulièrement nos plus vifs remerciements à **Madame Douae El Hila**, pour son encadrement, sa disponibilité, sa patience et ses précieux conseils tout au long de la réalisation de ce projet. Son soutien, ses orientations et ses remarques constructives nous ont permis de surmonter les difficultés rencontrées et d'améliorer la qualité de notre travail. Nous lui sommes profondément reconnaissants pour le temps qu'elle nous a consacré ainsi que pour la confiance qu'elle nous a accordée.

Nous tenons également à remercier l'ensemble des enseignants de l'École Nationale des Sciences Appliquées d'Oujda, qui ont contribué à notre formation académique et nous ont transmis les connaissances et les compétences nécessaires à la réalisation de ce projet. Grâce à leur dévouement et à la qualité de leur enseignement, nous avons pu acquérir de solides bases dans le domaine de l'informatique et du développement logiciel.

Nos remerciements s'adressent également à nos camarades de promotion pour leur esprit de collaboration, leurs échanges constructifs et leur aide précieuse tout au long de ce travail.

Enfin, nous exprimons notre profonde gratitude à nos familles et à nos proches pour leur soutien moral, leur patience, leurs encouragements constants et leur confiance en nos capacités. Leur présence et leur motivation ont été une source d'inspiration et de persévérance durant toute la période de réalisation de ce projet.

À toutes les personnes qui ont participé, directement ou indirectement, à l'aboutissement de ce projet de gestion d'une clinique, nous adressons nos plus sincères remerciements et notre profonde reconnaissance.

Table des matières

Remerciements	1
Table des figures	4
Liste des tableaux	5
Introduction	6
1 Présentation du projet	7
1.1 Description de l'application	7
1.2 Besoins fonctionnels	7
1.3 Besoins non fonctionnels	7
2 Conception	9
2.1 Architecture MVC	9
2.2 Diagramme de classes UML	10
2.3 Diagramme de cas d'utilisation	11
3 Environnement de travail	12
3.1 Outils et technologies utilisés	12
3.2 Structure du projet	12
4 Répartition des tâches	14
4.1 Tâches de Souaad Marjij	14
4.2 Tâches de Rayhane Mokhtari	14
5 Explication du code	15
5.1 Connexion à la base de données – <code>Database.java</code>	15
5.2 Les classes modèles	15
5.2.1 Patient	15
5.2.2 RendezVous	16
5.3 Les classes DAO	17
5.3.1 PatientDAO	17

5.3.2	RendezVousDAO	19
5.4	Les contrôleurs	22
5.4.1	MainController	22
5.4.2	PatientController	22
5.4.3	RendezVousController	23
5.4.4	DashboardController	24
5.5	Les fichiers FXML	24
5.6	Fonctionnalité spécifique au projet	24
6	Interfaces de l'application	26
6.1	Page de connexion	26
6.2	Interface de gestion des patients	27
6.3	Interface de gestion des rendez-vous	27
	Conclusion	29
	Références	30

Table des figures

2.1	Architecture MVC de l'application	9
2.2	Diagramme de classes UML	10
2.3	Diagramme de cas d'utilisation	11
6.1	Page de connexion – Gestion Clinique	26
6.2	Interface de gestion des patients	27
6.3	Interface de gestion des rendez-vous	28

Liste des tableaux

4.1	Tâches réalisées par Souaad Marjij	14
4.2	Tâches réalisées par Rayhane Mokhtari	14
5.1	Attributs de la classe Patient	16
5.2	Attributs de la classe RendezVous	16

Introduction

Dans le domaine médical, la bonne gestion des informations des patients et des rendez-vous est essentielle pour assurer un service de qualité et une meilleure organisation au sein d'une clinique. Une gestion manuelle de ces informations peut entraîner des erreurs, des pertes de données et des difficultés dans le suivi des patients.

Afin de remédier à ces problèmes, nous avons développé une application de **gestion d'une clinique** ayant pour objectif principal de faciliter l'administration des patients et la planification des rendez-vous. Cette application permet d'enregistrer, de modifier, de supprimer et de consulter les informations relatives aux patients ainsi qu'à leurs rendez-vous de manière simple et efficace.

Pour la réalisation de ce projet, nous avons utilisé les technologies **Java** et **JavaFX** pour la création de l'interface graphique, ainsi que **MySQL** pour la gestion de la base de données. Ce projet nous a permis de mettre en pratique les connaissances acquises durant notre formation en programmation orientée objet, en conception de bases de données et en développement d'applications de gestion.

Ainsi, cette application constitue une solution informatique visant à améliorer l'organisation et le suivi des activités essentielles d'une clinique.

Chapitre 1

Présentation du projet

1.1 Description de l'application

Le projet **Gestion d'une Clinique** est une application de bureau développée en **Java**, avec une interface graphique réalisée à l'aide de **JavaFX** et une base de données **MySQL**. Son objectif est de faciliter la gestion des patients et des rendez-vous au sein d'une clinique grâce à une interface simple et intuitive.

1.2 Besoins fonctionnels

L'application offre les fonctionnalités principales suivantes :

- **Gestion des patients** : ajout, modification, suppression et affichage de la liste des patients enregistrés.
- **Gestion des rendez-vous** : ajout, modification, suppression et affichage de la liste des rendez-vous programmés.
- **Authentification** : accès sécurisé à l'application via une page de connexion.
- **Navigation** : basculement entre les modules Patients et Rendez-vous depuis le menu latéral.

1.3 Besoins non fonctionnels

- **Performance** : chargement rapide des données depuis la base MySQL grâce à des requêtes JDBC optimisées.
- **Intégrité des données** : utilisation de `PreparedStatement` pour éviter les injections SQL et garantir la cohérence des données.
- **Ergonomie** : interface intuitive développée en JavaFX avec une mise en page claire et des boutons d'action explicitement nommés.

- **Maintenabilité** : architecture MVC et pattern DAO assurant une séparation claire des responsabilités et facilitant les évolutions futures.
- **Compatibilité** : application compatible avec JDK 17 et JavaFX SDK 17 sous Windows.

Chapitre 2

Conception

2.1 Architecture MVC

L'application repose sur l'architecture **MVC** (Modèle-Vue-Contrôleur), qui permet de séparer les responsabilités en trois couches distinctes :

- **Modèle (Model)** : contient les classes métier `Patient` et `RendezVous`, ainsi que les classes DAO (`PatientDAO`, `RendezVousDAO`) qui gèrent l'accès à la base de données MySQL via JDBC.
- **Vue (View)** : constituée des fichiers FXML (`patient.fxml`, `rendezvous.fxml`, `MainView.fxml`, `LoginView.fxml`) qui définissent la structure visuelle des interfaces, et du fichier `style.css` qui gère leur apparence.
- **Contrôleur (Controller)** : regroupe les classes `PatientController`, `RendezVousController`, `MainController` et `LoginController`, qui traitent les actions de l'utilisateur, communiquent avec la couche DAO et mettent à jour les vues en conséquence.

La figure suivante illustre l'organisation de cette architecture au sein du projet :

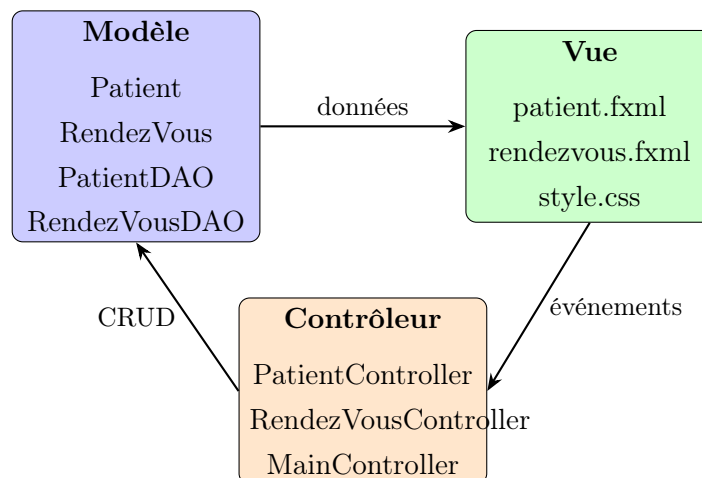


FIGURE 2.1 – Architecture MVC de l'application

2.2 Diagramme de classes UML

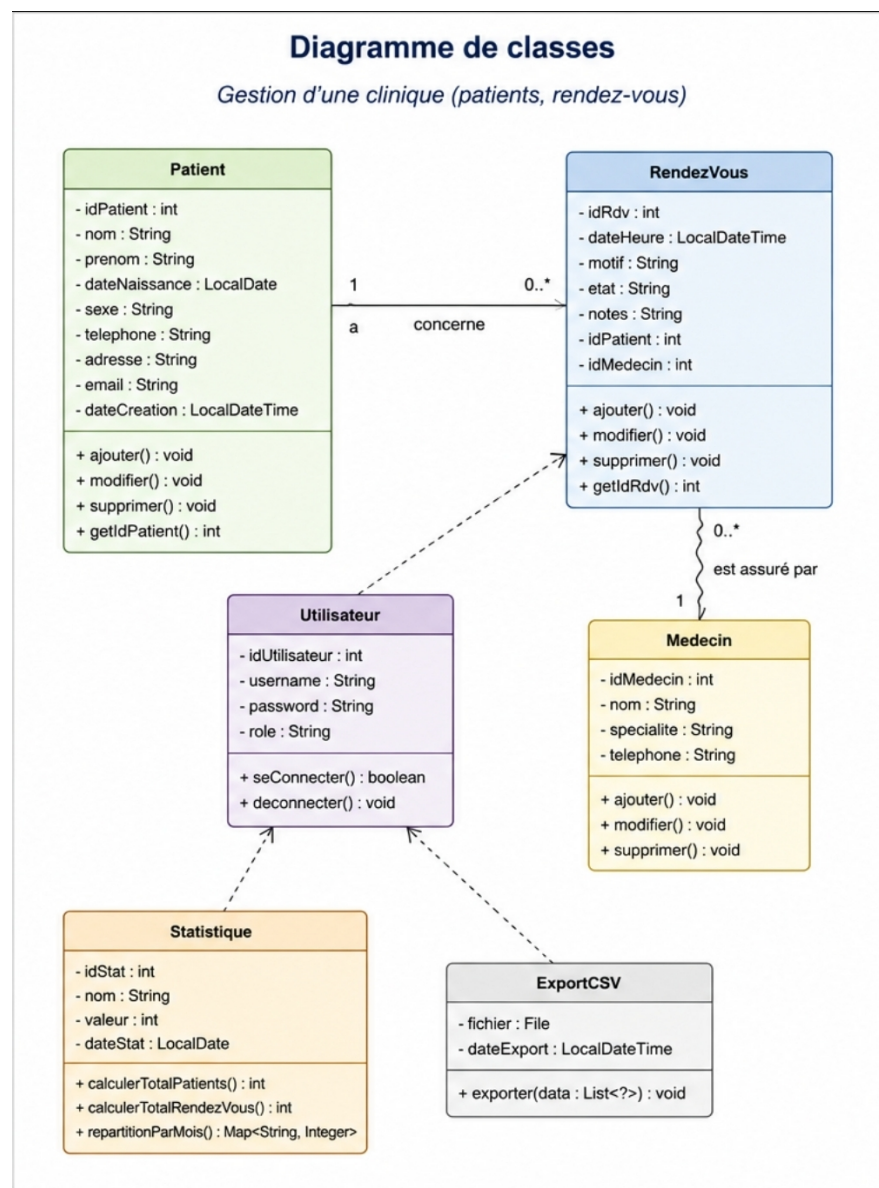


FIGURE 2.2 – Diagramme de classes UML

2.3 Diagramme de cas d'utilisation

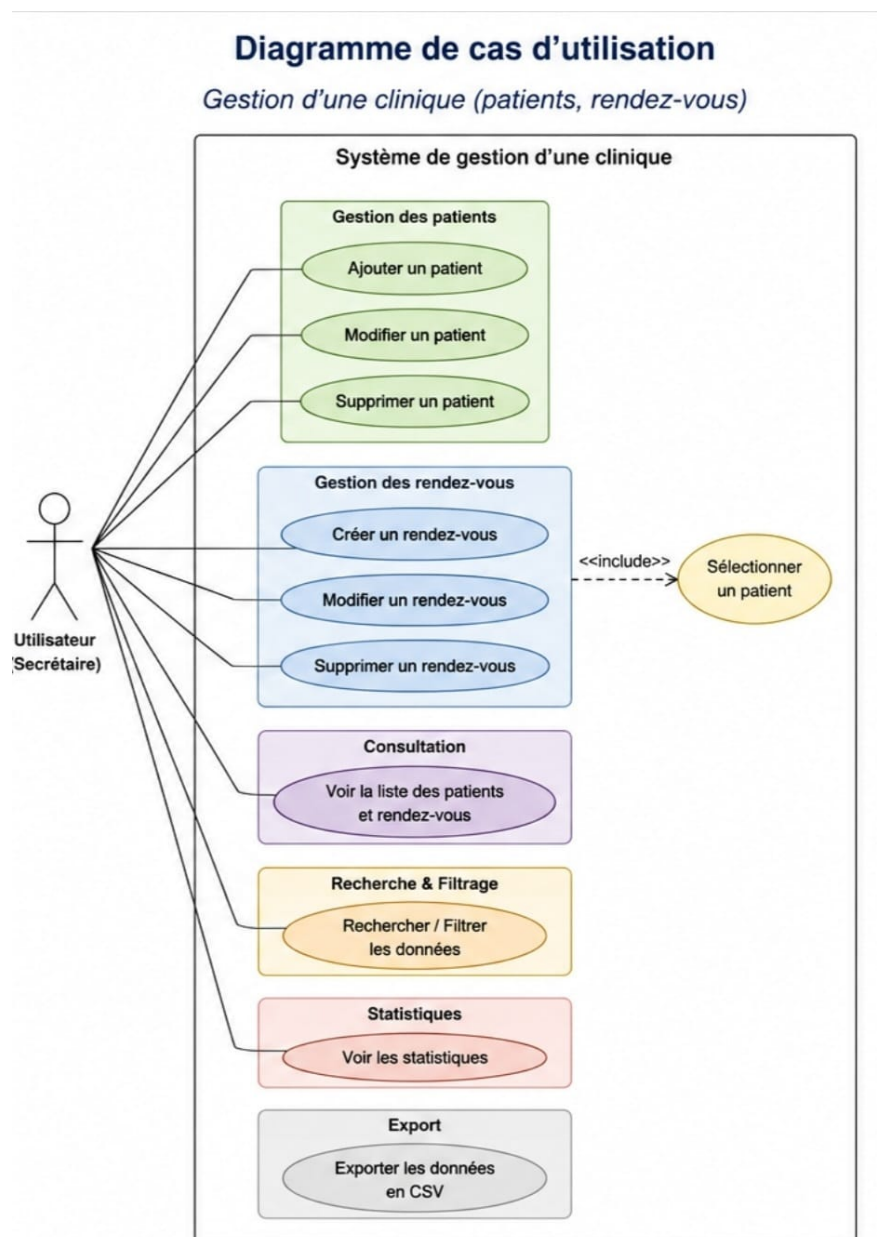


FIGURE 2.3 – Diagramme de cas d'utilisation

Chapitre 3

Environnement de travail

3.1 Outils et technologies utilisés

- **Langage** : Java 17
- **Framework IHM** : JavaFX SDK 17
- **Base de données** : MySQL
- **IDE** : Visual Studio Code
- **Gestion de version** : Git / GitHub
- **Outil UML** : draw.io

3.2 Structure du projet

```
projet/  
|-- .vscode/  
|-- bin/  
|-- demonstration/  
|   |-- Rapport_IHM.pdf  
|   -- DEMO.mp4  
|-- lib/  
|   -- mysql-connector-j-9.7.0.jar  
|-- src/  
|   |-- application/  
|   |   |-- LoginController.java  
|   |   -- Main.java  
|   |-- controller/  
|   |   |-- DashboardController.java  
|   |   |-- MainController.java  
|   |   |-- PatientController.java
```

```
| | -- RendezVousController.java
| |-- css/
| | -- style.css
| |-- dao/
| | |-- Database.java
| | |-- PatientDAO.java
| | -- RendezVousDAO.java
| |-- model/
| | |-- Patient.java
| | -- RendezVous.java
| -- view/
|     |-- DashboardView.fxml
|     |-- LoginView.fxml
|     |-- MainView.fxml
|     |-- patient.fxml
|     -- rendezvous.fxml
-- schema.sql
```

Chapitre 4

Répartition des tâches

4.1 Tâches de Souaad Marjij

Tâche réalisée	Fichier(s) concerné(s)	Statut
Création de la base de données	schema.sql	Terminé
Connexion JDBC	dao/Database.java	Terminé
DAO Patient (CRUD)	dao/PatientDAO.java	Terminé
DAO RendezVous (CRUD)	dao/RendezVousDAO.java	Terminé
Classes modèles	model/Patient.java, model/RendezVous.java	Terminé

TABLE 4.1 – Tâches réalisées par Souaad Marjij

4.2 Tâches de Rayhane Mokhtari

Tâche réalisée	Fichier(s) concerné(s)	Statut
Interface principale	view/MainView.fxml	Terminé
Interface gestion patients	view/patient.fxml	Terminé
Interface gestion rendez-vous	view/rendezvous.fxml	Terminé
Page de connexion	view/LoginView.fxml	Terminé
Contrôleurs JavaFX	controller/PatientController.java, controller/RendezVousController.java	Terminé
Style CSS	css/style.css	Terminé

TABLE 4.2 – Tâches réalisées par Rayhane Mokhtari

Chapitre 5

Explication du code

5.1 Connexion à la base de données – Database.java

```
1 public class Database {  
2     private static final String URL = "jdbc:mysql://localhost:3308 /  
   ma_bd";  
3     private static final String USER = "root";  
4     private static final String PASSWORD = "";  
5  
6     public static Connection getConnection() throws SQLException {  
7         return DriverManager.getConnection(URL, USER, PASSWORD);  
8     }  
9 }
```

Listing 5.1 – Connexion à la base de données

5.2 Les classes modèles

Les classes modèles représentent les entités métier de l'application. Elles suivent le pattern JavaBean : attributs privés accessibles via des getters et setters, et deux constructeurs (vide et paramétré).

5.2.1 Patient

La classe `Patient` modélise un patient de la clinique avec les attributs suivants :

Attribut	Type	Description
id	int	Identifiant unique (clé primaire)
nom	String	Nom de famille du patient
prenom	String	Prénom du patient
sexe	String	Sexe (M/F)
dateNaissance	LocalDate	Date de naissance
telephone	String	Numéro de téléphone
adresse	String	Adresse du patient

TABLE 5.1 – Attributs de la classe Patient

```

1 public Patient(int id, String nom, String prenom,
2               String sexe, LocalDate dateNaissance,
3               String telephone, String adresse) {
4     this.id = id;
5     this.nom = nom;
6     this.prenom = prenom;
7     this.sexe = sexe;
8     this.dateNaissance = dateNaissance;
9     this.telephone = telephone;
10    this.adresse = adresse;
11 }

```

Listing 5.2 – Constructeur paramétré – Patient

5.2.2 RendezVous

La classe `RendezVous` modélise un rendez-vous médical et est liée à un patient via la clé étrangère `patientId`.

Attribut	Type	Description
id	int	Identifiant unique du rendez-vous
patientId	int	Référence vers le patient concerné
dateRdv	LocalDate	Date du rendez-vous
heure	String	Heure du rendez-vous
motif	String	Motif de la consultation
urgence	int	Niveau d'urgence (1 à 3)
statut	String	État du RDV (planifié, annulé...)

TABLE 5.2 – Attributs de la classe RendezVous

```

1 public RendezVous(int id, int patientId, LocalDate dateRdv,
2                  String heure, String motif,

```

```
3         int urgence, String statut) {
4     this.id = id;
5     this.patientId = patientId;
6     this.dateRdv = dateRdv;
7     this.heure = heure;
8     this.motif = motif;
9     this.urgence = urgence;
10    this.statut = statut;
11 }
```

Listing 5.3 – Constructeur paramétré – RendezVous

5.3 Les classes DAO

Le pattern DAO (Data Access Object) permet de séparer la logique d'accès à la base de données de la logique métier. Chaque classe DAO gère les opérations CRUD (Create, Read, Update, Delete) pour une entité donnée, en utilisant JDBC pour communiquer avec la base de données MySQL.

5.3.1 PatientDAO

La classe PatientDAO assure la persistance des données des patients via quatre méthodes principales.

Ajout d'un patient

La méthode ajouter() insère un nouveau patient dans la table `patients` à l'aide d'un PreparedStatement pour éviter les injections SQL.

```
1 public void ajouter(Patient p) {
2     Connection conn = Database.getConnection();
3     String sql = "INSERT INTO patients (nom, prenom, sexe, "
4                 + "date_naissance, telephone, adresse) "
5                 + "VALUES (?, ?, ?, ?, ?, ?)";
6     try (PreparedStatement ps = conn.prepareStatement(sql)) {
7         ps.setString(1, p.getNom());
8         ps.setString(2, p.getPrenom());
9         ps.setString(3, p.getSexe());
10        ps.setDate(4, Date.valueOf(p.getDateNaissance()));
11        ps.setString(5, p.getTelephone());
12        ps.setString(6, p.getAdresse());
13        ps.executeUpdate();
14    } catch (SQLException e) {
15        System.err.println("Erreur ajout patient : " + e.getMessage());
16    }
17 }
```

```

16     }
17 }

```

Listing 5.4 – Méthode ajouter() – PatientDAO

Récupération de tous les patients

La méthode `getAll()` exécute une requête `SELECT *` et retourne une liste d'objets `Patient` construits à partir du `ResultSet`.

```

1 public List<Patient> getAll() {
2     List<Patient> liste = new ArrayList<>();
3     Connection conn = Database.getConnection();
4     String sql = "SELECT * FROM patients";
5     try (Statement st = conn.createStatement();
6         ResultSet rs = st.executeQuery(sql)) {
7         while (rs.next()) {
8             Patient p = new Patient();
9             p.setId(rs.getInt("id"));
10            p.setNom(rs.getString("nom"));
11            p.setPrenom(rs.getString("prenom"));
12            p.setSexe(rs.getString("sexe"));
13            p.setDateNaissance(
14                rs.getDate("date_naissance").toLocalDate());
15            p.setTelephone(rs.getString("telephone"));
16            p.setAdresse(rs.getString("adresse"));
17            liste.add(p);
18        }
19    } catch (SQLException e) {
20        System.err.println("Erreur liste patients : " + e.getMessage());
21    }
22    return liste;
23 }

```

Listing 5.5 – Méthode getAll() – PatientDAO

Modification d'un patient

La méthode `modifier()` met à jour tous les champs d'un patient existant en filtrant par son identifiant `id`.

```

1 public void modifier(Patient p) {
2     Connection conn = Database.getConnection();
3     String sql = "UPDATE patients SET nom=?, prenom=?, sexe=?, "
4                 + "date_naissance=?, telephone=?, adresse=? "
5                 + "WHERE id=?";
6     try (PreparedStatement ps = conn.prepareStatement(sql)) {

```

```
7      ps.setString(1, p.getNom());
8      ps.setString(2, p.getPrenom());
9      ps.setString(3, p.getSexe());
10     ps.setDate(4, Date.valueOf(p.getDateNaissance()));
11     ps.setString(5, p.getTelephone());
12     ps.setString(6, p.getAdresse());
13     ps.setInt(7, p.getId());
14     ps.executeUpdate();
15 } catch (SQLException e) {
16     System.err.println("Erreur modification : " + e.getMessage());
17 }
18 }
```

Listing 5.6 – Méthode modifier() – PatientDAO

Suppression d'un patient

La méthode `supprimer()` supprime un patient de la base de données en utilisant son identifiant.

```
1 public void supprimer(int id) {
2     Connection conn = Database.getConnection();
3     String sql = "DELETE FROM patients WHERE id=?";
4     try (PreparedStatement ps = conn.prepareStatement(sql)) {
5         ps.setInt(1, id);
6         ps.executeUpdate();
7     } catch (SQLException e) {
8         System.err.println("Erreur suppression : " + e.getMessage());
9     }
10 }
```

Listing 5.7 – Méthode supprimer() – PatientDAO

5.3.2 RendezVousDAO

La classe `RendezVousDAO` gère la persistance des rendez-vous en base de données. Elle manipule la table `rendezvous` qui contient les champs : `patient_id`, `date_rdv`, `heure`, `motif`, `urgence` et `statut`.

Ajout d'un rendez-vous

La méthode `ajouter()` insère un nouveau rendez-vous en associant l'identifiant du patient via la clé étrangère `patient_id`.

```
1 public void ajouter(RendezVous r) {
2     Connection conn = Database.getConnection();
3     String sql = "INSERT INTO rendezvous (patient_id, date_rdv, "
```

```

4         + "heure, motif, urgence, statut) "
5         + "VALUES (?, ?, ?, ?, ?, ?)";
6     try (PreparedStatement ps = conn.prepareStatement(sql)) {
7         ps.setInt(1, r.getPatientId());
8         ps.setDate(2, Date.valueOf(r.getDateRdv()));
9         ps.setString(3, r.getHeure());
10        ps.setString(4, r.getMotif());
11        ps.setInt(5, r.getUrgence());
12        ps.setString(6, r.getStatut());
13        ps.executeUpdate();
14    } catch (SQLException e) {
15        System.err.println("Erreur ajout RDV : " + e.getMessage());
16    }
17 }

```

Listing 5.8 – Méthode ajouter() – RendezVousDAO

Récupération de tous les rendez-vous

La méthode `getAll()` retourne la liste complète des rendez-vous enregistrés, en reconstruisant chaque objet `RendezVous` depuis le `ResultSet`.

```

1 public List<RendezVous> getAll() {
2     List<RendezVous> liste = new ArrayList<>();
3     Connection conn = Database.getConnection();
4     String sql = "SELECT * FROM rendezvous";
5     try (Statement st = conn.createStatement();
6         ResultSet rs = st.executeQuery(sql)) {
7         while (rs.next()) {
8             RendezVous r = new RendezVous();
9             r.setId(rs.getInt("id"));
10            r.setPatientId(rs.getInt("patient_id"));
11            r.setDateRdv(rs.getDate("date_rdv").toLocalDate());
12            r.setHeure(rs.getString("heure"));
13            r.setMotif(rs.getString("motif"));
14            r.setUrgence(rs.getInt("urgence"));
15            r.setStatut(rs.getString("statut"));
16            liste.add(r);
17        }
18    } catch (SQLException e) {
19        System.err.println("Erreur liste RDV : " + e.getMessage());
20    }
21    return liste;
22 }

```

Listing 5.9 – Méthode getAll() – RendezVousDAO

Modification d'un rendez-vous

La méthode `modifier()` met à jour tous les champs d'un rendez-vous existant identifié par son `id`.

```
1 public void modifier(RendezVous r) {
2     Connection conn = Database.getConnection();
3     String sql = "UPDATE rendezvous SET patient_id=?, date_rdv=?, "
4                 + "heure=?, motif=?, urgence=?, statut=? WHERE id=?";
5     try (PreparedStatement ps = conn.prepareStatement(sql)) {
6         ps.setInt(1, r.getPatientId());
7         ps.setDate(2, Date.valueOf(r.getDateRdv()));
8         ps.setString(3, r.getHeure());
9         ps.setString(4, r.getMotif());
10        ps.setInt(5, r.getUrgence());
11        ps.setString(6, r.getStatut());
12        ps.setInt(7, r.getId());
13        ps.executeUpdate();
14    } catch (SQLException e) {
15        System.err.println("Erreur modification RDV : " + e.getMessage());
16    }
17 }
```

Listing 5.10 – Méthode `modifier()` – `RendezVousDAO`

Suppression d'un rendez-vous

La méthode `supprimer()` supprime un rendez-vous de la base de données à partir de son identifiant.

```
1 public void supprimer(int id) {
2     Connection conn = Database.getConnection();
3     String sql = "DELETE FROM rendezvous WHERE id=?";
4     try (PreparedStatement ps = conn.prepareStatement(sql)) {
5         ps.setInt(1, id);
6         ps.executeUpdate();
7     } catch (SQLException e) {
8         System.err.println("Erreur suppression RDV : " + e.getMessage());
9     }
10 }
```

Listing 5.11 – Méthode `supprimer()` – `RendezVousDAO`

5.4 Les contrôleurs

L'application suit l'architecture MVC : chaque contrôleur JavaFX gère une vue FXML et communique avec la couche DAO pour accéder aux données. L'application dispose de quatre contrôleurs principaux.

5.4.1 MainController

`MainController` est le contrôleur racine de l'application. Il gère un `BorderPane` principal et assure la navigation entre les différentes vues en chargeant dynamiquement les fichiers FXML dans la zone centrale.

```
1 private void load(String fxml) {
2     try {
3         FXMLLoader loader = new FXMLLoader(
4             getClass().getResource("/view/" + fxml));
5         Parent root = loader.load();
6         mainPane.setCenter(root);
7     } catch (Exception e) {
8         System.err.println("Erreur chargement vue : " + fxml);
9     }
10 }
11
12 @FXML
13 public void openPatients() { load("patient.fxml"); }
14
15 @FXML
16 public void openRendezVous() { load("rendezvous.fxml"); }
```

Listing 5.12 – Navigation entre vues – `MainController`

5.4.2 PatientController

`PatientController` gère l'interface de gestion des patients. Il implémente `Initializable` et s'appuie sur `PatientDAO` pour les opérations CRUD. À l'initialisation, il configure les colonnes de la `TableView` et charge les données depuis la base. Un écouteur sur la sélection de la table remplit automatiquement le formulaire avec les données du patient sélectionné.

```
1 @Override
2 public void initialize(URL url, ResourceBundle resourceBundle) {
3     patientDAO = new PatientDAO();
4     cbSexe.setItems(FXCollections.observableArrayList("M", "F"));
5
6     colId.setCellValueFactory(new PropertyValueFactory<>("id"));
```

```

7     colNom.setCellValueFactory(new PropertyValueFactory<>("nom"));
8     colPrenom.setCellValueFactory(new PropertyValueFactory<>("prenom")
9 );
10    // ...
11    loadTableData();
12
13    tablePatients.getSelectionModel()
14        .selectedItemProperty()
15        .addListener((obs, old, newVal) -> {
16            if (newVal != null) { fillFields(newVal); }
17        });
18    }

```

Listing 5.13 – Initialisation et chargement – PatientController

Les trois actions principales sont déclenchées par les boutons de l'interface :

- `addPatient()` : crée un objet `Patient` à partir du formulaire et appelle `patientDAO.ajouter()`
- `updatePatient()` : met à jour le patient sélectionné via `patientDAO.modifier()`
- `deletePatient()` : supprime via `patientDAO.supprimer()` après vérification de la sélection

Une méthode `isInputValid()` vérifie que les champs obligatoires (nom, prénom, sexe, date de naissance) sont renseignés avant toute opération d'écriture.

5.4.3 RendezVousController

`RendezVousController` gère la planification des rendez-vous. Il utilise à la fois `RendezVousDAO` et `PatientDAO` pour charger la liste des patients dans un `ComboBox` au format `id - Nom Prénom`. Le niveau d'urgence est saisi via un `Slider` (valeur entière de 1 à 3). Le statut du rendez-vous peut prendre trois valeurs : *Prévu*, *Terminé* ou *Annulé*.

```

1 private void loadPatients() {
2     ObservableList<String> patients = FXCollections.
3     observableArrayList();
4     for (Patient p : patientDAO.getAll()) {
5         patients.add(p.getId() + " - " + p.getNom()
6                     + " " + p.getPrenom());
7     }
8     cbPatient.setItems(patients);
9 }
10 private int extractPatientId() {
11     String val = cbPatient.getValue();
12     return Integer.parseInt(val.split(" - ")[0]);
13 }

```

Listing 5.14 – Chargement des patients et extraction de l'id – RendezVousController

5.4.4 DashboardController

`DashboardController` gère la vue tableau de bord. Il affiche des statistiques générales de l'application, notamment le nombre total de patients via un `Label` (`lblPatients`).

5.5 Les fichiers FXML

Les interfaces graphiques sont définies en FXML et chargées dynamiquement par `MainController`. L'application contient trois fichiers FXML principaux :

- `main.fxml` : fenêtre principale avec un `BorderPane` ; la barre de navigation latérale contient les boutons *Patients* et *Rendez-vous* qui déclenchent le chargement dynamique des vues.
- `patient.fxml` : interface de gestion des patients, composée d'un formulaire de saisie (nom, prénom, sexe via `ComboBox`, date de naissance via `DatePicker`, téléphone, adresse) et d'une `TableView` affichant la liste complète des patients.
- `rendezvous.fxml` : interface de gestion des rendez-vous, avec un `ComboBox` de sélection du patient, un `DatePicker`, un champ heure, une `TextArea` pour le motif, un `Slider` pour le niveau d'urgence, et un `ComboBox` pour le statut.

5.6 Fonctionnalité spécifique au projet

Une fonctionnalité clé de l'application est la **gestion du niveau d'urgence des rendez-vous**. Lors de la planification d'un rendez-vous, le praticien attribue un niveau d'urgence (1 = normal, 2 = urgent, 3 = très urgent) via un `Slider` JavaFX. Cette valeur est persistée en base et permet de prioriser l'affichage des rendez-vous dans la `TableView`.

```
1 @FXML
2 public void ajouterRdv() {
3     if (isInputValid()) {
4         RendezVous r = new RendezVous(
5             0,
6             extractPatientId(),
7             dateRdv.getValue(),
8             txtHeure.getText(),
9             txtMotif.getText(),
10            (int) sliderUrgence.getValue(),
11            cbStatut.getValue()
12        );
13        rdvDAO.ajouter(r);
14        loadTableData();
15        clearFields();
16    }
```

```
17 }
```

Listing 5.15 – Création d’un rendez-vous avec niveau d’urgence

Chapitre 6

Interfaces de l'application

6.1 Page de connexion

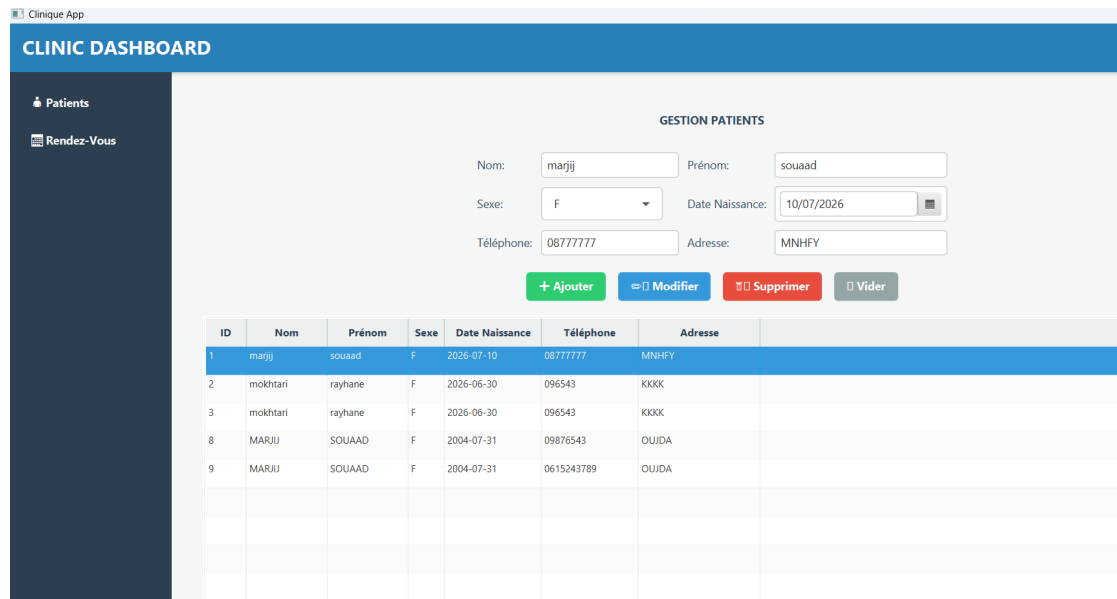
La page de connexion constitue le point d'entrée de l'application. Elle présente un formulaire sécurisé avec un champ nom d'utilisateur et un champ mot de passe, permettant l'accès à l'espace d'administration de la clinique.



FIGURE 6.1 – Page de connexion – Gestion Clinique

6.2 Interface de gestion des patients

L'interface de gestion des patients est composée de deux parties : un formulaire de saisie en haut (nom, prénom, sexe, date de naissance, téléphone, adresse) et une **TableView** en bas affichant la liste complète des patients enregistrés. La sélection d'une ligne dans le tableau remplit automatiquement le formulaire pour permettre la modification ou la suppression. Quatre boutons d'action sont disponibles : *Ajouter*, *Modifier*, *Supprimer* et *Vider*.



CLINIC DASHBOARD

Patients
Rendez-Vous

GESTION PATIENTS

Nom: marjij Prénom: souaad
Sexe: F Date Naissance: 10/07/2026
Téléphone: 08777777 Adresse: MNHEY

+ Ajouter Modifier Supprimer Vider

ID	Nom	Prénom	Sexe	Date Naissance	Téléphone	Adresse
1	marjij	souaad	F	2026-07-10	08777777	MNHEY
2	mokhtari	rayhane	F	2026-06-30	096543	KXXX
3	mokhtari	rayhane	F	2026-06-30	096543	KXXX
8	MARIJJI	SOUAAD	F	2004-07-31	09876543	OUIDA
9	MARIJJI	SOUAAD	F	2004-07-31	0615243789	OUIDA

FIGURE 6.2 – Interface de gestion des patients

6.3 Interface de gestion des rendez-vous

L'interface de gestion des rendez-vous permet de planifier, modifier et supprimer des rendez-vous médicaux. Elle propose une sélection du patient via un **ComboBox**, un **DatePicker** pour la date, un champ heure au format HH :mm, une zone de texte pour le motif, un **Slider** pour le niveau d'urgence (1 à 5), et un **ComboBox** pour le statut (Prévu, Terminé, Annulé). La **TableView** affiche l'ensemble des rendez-vous avec leurs informations détaillées.

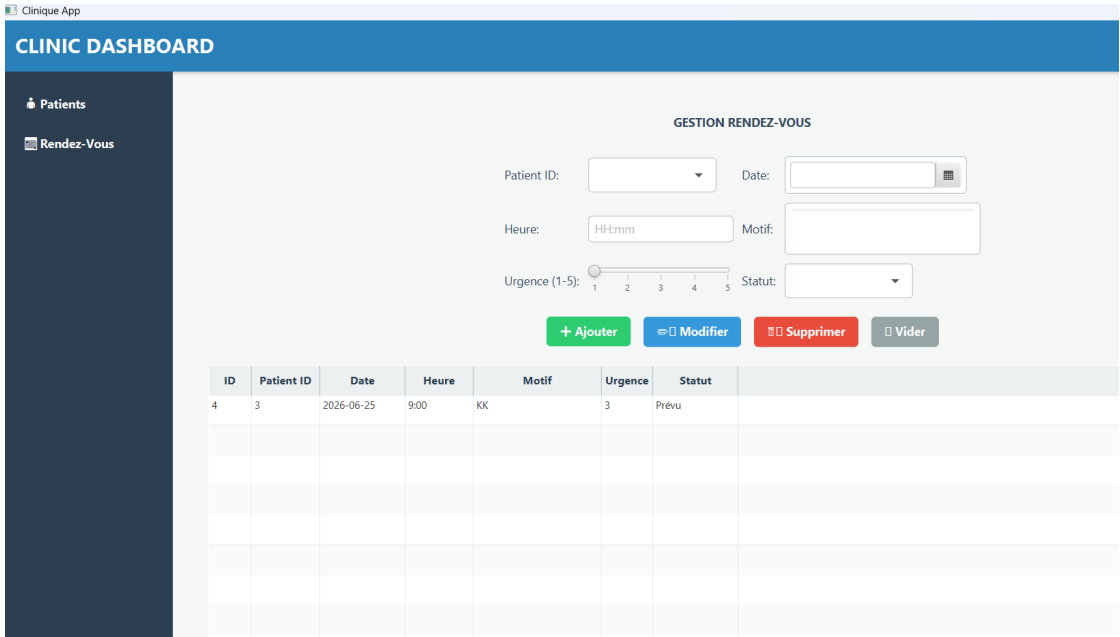


FIGURE 6.3 – Interface de gestion des rendez-vous

Conclusion

La réalisation de ce projet de gestion d'une clinique nous a permis de développer une application dédiée à la gestion des patients et des rendez-vous médicaux. En adoptant une architecture MVC et en appliquant le pattern DAO, nous avons pu produire une solution structurée, maintenable et extensible.

Ce projet nous a offert l'occasion de mettre en pratique nos connaissances en **Java**, **JavaFX** et **MySQL**, tout en approfondissant nos compétences en conception orientée objet, en modélisation UML et en développement d'interfaces graphiques.

Sur le plan technique, nous avons su surmonter plusieurs défis, notamment la configuration de l'environnement JavaFX, la mise en place de la couche JDBC et la gestion dynamique des vues via **FXMLLoader**.

En perspective, cette application pourrait être enrichie par l'ajout de fonctionnalités telles que la gestion des médecins, la génération de rapports PDF, ou encore un système de notifications pour les rendez-vous à venir. Elle constitue ainsi une première étape vers la création de solutions informatiques plus complètes dans le domaine de la santé.

Références

- Documentation officielle JavaFX : <https://openjfx.io>
- Documentation MySQL : <https://dev.mysql.com/doc/>
- Documentation Java 17 : <https://docs.oracle.com/en/java/javase/17/>
- Connecteur JDBC MySQL : <https://dev.mysql.com/downloads/connector/j/>
- Outil de modélisation UML draw.io : <https://www.drawio.com>
- Plateforme de gestion de code GitHub : <https://github.com/Marjij-Souaad/GESTION-D-UNE-CLINIQUE>